HW3 Combined Report: DQN and Its Variants

Course assignment: HW3 - DQN and its variants
Reference: Deep Reinforcement Learning in Action, Chapter 3 GridWorld examples
Environment family: 4x4 GridWorld with Player, Goal, Pit, and Wall layers

1. Overall Summary

This homework implements and compares several Deep Q-Network methods on three GridWorld modes:

- Static mode: the player, goal, pit, and wall positions are fixed.
- Player mode: the goal, pit, and wall are fixed, but the player starts randomly.
- Random mode: the player, goal, pit, and wall are all randomized every episode.

The state is encoded as four 4x4 one-hot layers: Player, Goal, Pit, and Wall. After flattening, each observation is a 64-dimensional vector. The action space has four actions: up, down, left, and right. Reaching the goal gives +10, falling into the pit gives -10, and each non-terminal move gives -1.

Summary of final results:

Task	Mode	Main method	Evaluation result
HW3-1	Static	Naive DQN + Replay Buffer	50/50 wins, 100% win rate
HW3-2	Player	Basic / Double / Dueling DQN	All reached 100% eval win rate
HW3-3	Random	PyTorch Lightning enhanced DQN	292/300 wins, 97.33% win rate
HW3-4	Random	Rainbow DQN	278/300 wins, 92.67% win rate

2. HW3-1: Naive DQN for Static Mode

Task requirement:
Implement or run a basic DQN for an easy environment, include an experience replay buffer, and write a short understanding report.

Implementation files:

- hw3_1/gridworld.py
- hw3_1/naive_dqn_static.py

Method:
The basic DQN uses a feed-forward Q-network. The network receives the flattened 64-dimensional state vector and outputs four Q-values, one for each action.

The TD target is:

```
y = reward + gamma * (1 - done) * max_a Q(next_state, a)
```

This is a naive DQN because the same online network is used to estimate both the current Q-value and the bootstrap target. No target network is used in HW3-1.

Experience replay:
The replay buffer stores transitions:

```
(state, action, reward, next_state, done)
```

Training samples random minibatches from the buffer instead of only using the newest transition. This reduces temporal correlation and improves sample reuse.

Run command:

```
python -m hw3_1.naive_dqn_static --episodes 1500 --eval-episodes 50 --seed 7
```

Result:

```
Evaluation episodes: 50
Wins: 50
Win rate: 100%
Average episode length: 7 moves
Example learned path: down, down, left, left, left, up, up
```

Saved outputs:

```
outputs/hw3_1_static_metrics.json
outputs/hw3_1_static_training.csv
outputs/hw3_1_static_dqn.pt
```

Understanding:
Static mode is simple because the optimal path is fixed. The main learning objective is to associate each state-action pair along the path with a high future return. The replay buffer is still useful because it makes training less dependent on the exact order of transitions collected during an episode.

3. HW3-2: Enhanced DQN Variants for Player Mode

Task requirement:
Implement and compare Double DQN and Dueling DQN in player mode, focusing on how they improve upon the basic DQN approach.

Implementation files:

```
hw3_1/gridworld.py
hw3_2/player_mode_variants.py
```

Environment:
Player mode keeps the goal, pit, and wall fixed, but the player starts from a random valid cell. This is harder than static mode because the agent must learn a policy for many possible starting states.

Compared methods:

Basic DQN:

```
y = r + gamma * max_a Q_online(s', a)
```

Basic DQN is simple but can overestimate action values because the same network selects and evaluates the max next action.

Double DQN:

```
a* = argmax_a Q_online(s', a)
y = r + gamma * Q_target(s', a*)
```

Double DQN reduces overestimation bias by separating action selection and action evaluation.

Dueling DQN:

```
Q(s,a) = V(s) + A(s,a) - mean_a A(s,a)
```

Dueling DQN separates state value from action advantage. This helps the model learn whether a state is generally good even when the differences between actions are small.

Run command:

```
python -m hw3_2.player_mode_variants --episodes 2200 --eval-repeats 10 --seed 11
```

Result:
The evaluation tested all 13 valid player starting positions for 10 repeats, giving 130 evaluation episodes per method.

Method	Final train success	First episode rolling success >= 90%	Eval win rate	Avg return	Avg length
Basic DQN	100%	506	100%	6.615	4.385
Double DQN	100%	472	100%	6.615	4.385
Dueling DQN	100%	507	100%	6.615	4.385

Saved outputs:

```
outputs/hw3_2_comparison_summary.csv
outputs/hw3_2_comparison_metrics.json
outputs/hw3_2_basic_dqn.pt
outputs/hw3_2_double_dqn.pt
outputs/hw3_2_dueling_dqn.pt
```

Interpretation:
All three methods solved this small player-mode GridWorld. The final evaluation win rate does not separate the methods, so the more useful comparison is learning speed. Double DQN reached the 90% rolling training success threshold earliest in this run, consistent with its purpose of reducing over-optimistic max-Q targets.

4. HW3-3: Enhanced DQN for Random Mode with PyTorch Lightning

Task requirement:
Convert the DQN model from PyTorch to either Keras or PyTorch Lightning. Add training techniques to stabilize or improve learning.

Implementation files:

```
hw3_1/gridworld.py
hw3_3/lightning_random_dqn.py
```

Framework conversion:
This task uses PyTorch Lightning. The DQN agent is implemented as a LightningModule. Since reinforcement learning does not use a fixed supervised dataset, the Lightning DataLoader yields dummy episode indices and each training_step runs one full environment episode.

Random mode:
In random mode, the player, goal, pit, and wall are all sampled randomly every episode without overlap. The implementation also checks that the goal is reachable from the player without crossing the wall or pit.

Training techniques:

- Convolutional state encoder for the 4x4 board structure
- Dueling Q-network
- Double DQN target computation
- Target network synchronization
- Experience replay
- Huber loss
- Gradient clipping
- AdamW optimizer
- Cosine learning rate scheduling
- Epsilon-greedy exploration schedule
- Manhattan-distance reward shaping

Why these help:
Double DQN and the target network reduce unstable bootstrap targets. The dueling architecture improves representation learning. Huber loss and gradient clipping reduce the effect of large TD errors. The learning rate scheduler makes late-stage training smoother. Reward shaping gives the agent a small signal when it moves closer to the goal.

Run command:

```
python -m hw3_3.lightning_random_dqn --episodes 5000 --eval-episodes 300 --seed 23
```

Result:

```
Training episodes: 5000
Final rolling training success over last 100 episodes: 99%
Evaluation episodes: 300
Wins: 292
Win rate: 97.33%
Average raw return: 6.757
Average episode length: 3.95 moves
```

Saved outputs:

```
outputs/hw3_3_lightning_random_metrics.json
outputs/hw3_3_lightning_random_training.csv
outputs/hw3_3_lightning_random_eval.csv
outputs/hw3_3_lightning_random_dqn.ckpt
outputs/hw3_3_lightning_random_dqn.pt
```

Interpretation:
Random mode is harder because the optimal path changes every episode. The convolutional encoder helped preserve spatial structure, while Double DQN and the target network stabilized target values. The final policy solved most unseen random boards and reached the goal in about four moves on average.

==========
5. HW3-4 Bonus: Rainbow DQN for Random Mode
==========

Task requirement:
Use Rainbow DQN to solve Random Mode GridWorld.

Implementation files:

    hw3_4/rainbow_random_dqn.py

Rainbow components:

    - Double DQN
    - Dueling network
    - Prioritized Experience Replay
    - 5-step returns
    - C51 distributional value function
    - NoisyLinear exploration

C51 setup:
The model predicts a categorical return distribution instead of a single Q-value. The support is:

    v_min = -50
    v_max = 15
    atoms = 51

This range covers strongly negative timeout episodes and positive goal rewards.

Additional stabilizers:

    - Convolutional board encoder
    - Target network synchronization
    - AdamW optimizer
    - Cosine learning rate scheduler
    - Gradient clipping
    - Manhattan-distance reward shaping
    - Importance-sampling weights for PER
    - Invalid-action mask for wall and out-of-bounds moves
    - Auxiliary expected-Q Huber loss

The invalid-action mask only removes moves that would go outside the board or hit the wall. It does not block pit actions, so avoiding the pit still has to be learned from reward.

Run command:

    python -m hw3_4.rainbow_random_dqn --episodes 7000 --eval-episodes 300 --seed 31

Result:

    Training episodes: 7000
    Final rolling training success over last 100 episodes: 89%
    Evaluation episodes: 300
    Wins: 278
    Win rate: 92.67%
    Average raw return: 5.487
    Average episode length: 4.407 moves

Saved outputs:

    outputs/hw3_4_rainbow_random_metrics.json

outputs/hw3_4_rainbow_random_training.csv
outputs/hw3_4_rainbow_random_eval.csv
outputs/hw3_4_rainbow_random_dqn.pt

Interpretation:
Rainbow DQN is heavier than the previous DQN variants because it learns a categorical return distribution and maintains replay priorities. On this small GridWorld, the most useful parts were n-step returns, Double DQN targets, the convolutional dueling architecture, and invalid-action masking. PER and C51 were implemented, but the final version needed an auxiliary expected-Q Huber loss to make learning stable on this small environment.

==
6. Cross-Task Comparison
==

The environment difficulty increases from HW3-1 to HW3-3/HW3-4:

Static mode is easiest because there is one fixed path.
Player mode is harder because the start position changes.
Random mode is hardest because every object changes position.

Final comparison:

Task	Method	Mode	Eval win rate	Avg length
HW3-1	Naive DQN + Replay	Static	100%	7.000
HW3-2	Basic DQN	Player	100%	4.385
HW3-2	Double DQN	Player	100%	4.385
HW3-2	Dueling DQN	Player	100%	4.385
HW3-3	Lightning enhanced DQN	Random	97.33%	3.950
HW3-4	Rainbow DQN	Random	92.67%	4.407

The random-mode enhanced DQN in HW3-3 achieved the highest random-mode evaluation win rate. Rainbow DQN was more complex and implemented more advanced components, but on this small GridWorld the full Rainbow setup needed extra stabilization and did not outperform the simpler enhanced DQN. This is reasonable because Rainbow methods are usually most beneficial in larger, noisier environments where distributional learning, prioritized replay, and noisy exploration have more room to help.

==
7. Conclusion
==

This homework shows how DQN can be improved step by step:

HW3-1 demonstrates the core idea of Q-learning with a neural network and replay buffer.
HW3-2 shows how Double DQN and Dueling DQN improve the basic DQN formulation.
HW3-3 shows how framework organization and training techniques improve stability in random mode.
HW3-4 implements Rainbow DQN and demonstrates the interaction of multiple advanced DQN components.

The best final result for Random Mode GridWorld was the PyTorch Lightning enhanced DQN from HW3-3, with a 97.33% evaluation win rate. The Rainbow DQN bonus implementation reached 92.67%, showing that the full Rainbow design works but requires careful tuning on small environments.